AceCoder: An Effective Prompting Technique Specialized in Code Generation

JIA LI (he/him/his), YUNFEI ZHAO, YONGMIN LI, GE LI, and ZHI JIN, Key Laboratory of High Confidence Software Technologies, Ministry of Education, Peking University, Beijing, China and School of Computer Science, Peking University, Beijing, China

Large language models (LLMs) have shown great success in code generation. LLMs take as the input a prompt and output the code. How to make prompts (i.e., *Prompting Techniques*) is a key question. Existing prompting techniques are designed for natural language generation and have low accuracy in code generation.

In this article, we propose a new prompting technique named AceCoder. Our motivation is that code generation meets two unique challenges (i.e., requirement understanding and code implementation). AceCoder contains two novel mechanisms (i.e., guided code generation and example retrieval) to solve these challenges. ❶ Guided code generation asks LLMs first to analyze requirements and output an intermediate preliminary (e.g., test cases). The preliminary clarifies requirements and tells LLMs *"what to write."* ❷ Example retrieval selects similar programs as examples in prompts, which provide lots of relevant content (e.g., algorithms, APIs) and teach LLMs *"how to write."* We apply AceCoder to four LLMs (e.g., GPT-3.5, CodeGeeX) and evaluate it on three public benchmarks using the Pass@k. Results show that AceCoder can significantly improve the performance of LLMs on code generation. *In terms of Pass@1, AceCoder outperforms the SOTA baseline by up to 56.4% in MBPP, 70.7% in MBJP, and 88.4% in MBJSP.* AceCoder is effective in LLMs with different sizes (i.e., 6B–13B) and different languages (i.e., Python, Java, and JavaScript). Human evaluation shows human developers prefer programs from AceCoder.

CCS Concepts: • **Computing methodologies** → *Neural networks*; *Natural language processing*; • **Software and its engineering** → **Automatic programming**;

Additional Key Words and Phrases: Code generation, large language models, prompting engineering

This research is supported by the National Natural Science Foundation of China (Nos. 62192731, 62152730), the National Key R & D Program (No. 2023YFB4503801), the National Natural Science Foundation of China (Nos. 62072007, 62192733, 61832009, 62192730), and the Major Program (JD) of Hubei Province (No.2023BAA024).

Authors' Contact Information: Jia Li, Key Laboratory of High Confidence Software Technologies, Ministry of Education, Peking University, Beijing, China and School of Computer Science, Peking University, Beijing, China; e-mail: lijia@stu.pku.edu.cn; Yunfei Zhao, Key Laboratory of High Confidence Software Technologies, Ministry of Education, Peking University, Beijing, China and School of Computer Science, Peking University, Beijing, China; e-mail: zhaoyunfei@pku.edu.cn; Yongmin Li, Key Laboratory of High Confidence Software Technologies, Ministry of Education, Peking University, Beijing, China and School of Computer Science, Peking University, Beijing, China; e-mail: liyongmin@pku.edu.cn; Ge Li (Corresponding author), Key Laboratory of High Confidence Software Technologies, Ministry of Education, Peking University, Beijing, China and School of Computer Science, Peking University, Beijing, China; e-mail: lige@pku.edu.cn; Zhi Jin, Key Laboratory of High Confidence Software Technologies, Ministry of Education, Peking University, Beijing, China and School of Computer Science, Peking University, Beijing, China; e-mail: zhijin@pku.edu.cn.

ACM 1557-7392/2024/11-ART204
https://doi.org/10.1145/3675395

ACM Reference format:
Jia Li, Yunfei Zhao, Yongmin Li, Ge Li, and Zhi Jin. 2024. AceCoder: An Effective Prompting Technique Specialized in Code Generation. *ACM Trans. Softw. Eng. Methodol.* 33, 8, Article 204 (November 2024), 26 pages. https://doi.org/10.1145/3675395

1 Introduction

Code generation aims to automatically generate the source code based on a natural language requirement. Recently, **large language models (LLMs)** have achieved **state-of-the-art (SOTA)** results on code generation [12, 13, 25, 30, 51]. LLMs do not require fine-tuning and take a prompt as input. A prompt consists of several examples (e.g., <requirement, code pairs>) and a new requirement. LLMs learn code generation from examples and analogously generate code for the new requirement.

The performance of In-Context Learning strongly relies on the prompt surface [50]. How to design prompts (i.e., prompting techniques) is still an open question. Existing prompting techniques (e.g., few-shot prompting [9] and **chain-of-thought (CoT)** prompting [48]) are designed for natural language generation and have low accuracy in code generation. For example, Codex with few-shot prompting only achieves 37.2% Pass@1[1] on a real-world benchmark—HumanEval [12]. Thus, exploring more advanced prompting techniques for code generation is necessary.

In this article, we propose a novel prompting technique specialized in code generation, named AceCoder**.** It significantly improves the performance of LLMs in code generation. Our motivation is that code generation aims to build a mapping from natural language requirements to source code. There are two unique challenges in this mapping, i.e., requirement understanding and code implementation. AceCoder proposes two novel mechanisms to alleviate two challenges. The details of AceCoder are shown as follows.

Challenge 1: Requirement Understanding. Understanding requirements is the starting point of code generation. In real-world programming problems, the requirement may be a brief purpose without specific details. For example, a requirement from a real-world benchmark—**Mostly Basic Programming Problems (MBPP)** [7] is `write a function to check if the triangle is isosceles or not`. Before writing code, we need to analyze the requirement and determine specific details, e.g., input–output formats, and possible exceptions.

Novelty 1: Guided Code Generation. To alleviate this challenge, we propose *guided code generation*. Our motivation is that human developers often use some software artifacts to assist in analyzing requirements. For example, in test-driven development [8], developers clarify requirements by designing test cases. It forces developers to think about details of requirements, e.g., input–output formats and boundary values. These test cases exactly define the requirement and tell developers *what to write*.

To implement the above process, we design a special prompt consisting of triple examples (i.e.,<requirement, preliminary, code>). A preliminary is a specific software artifact (e.g., test cases, APIs) for clarifying the requirement. Given a new requirement, based on the prompt, LLMs first output a preliminary and then generate code based on the preliminary. We illustrate the guided code generation in Section 2 and describe the details in Section 3.3.

Challenge 2: Code Implementation. After understanding the requirements, implementing the source code using a programming language is challenging. It requires LLMs to master related

[1]Pass@k (e.g., $k = 1$) is a widely used evaluation metric in code generation. It denotes the percentage of programs passing all test cases within models' outputs. The detailed definition of Pass@k can be found in Section 4.2.

grammar, algorithms, and libraries. Even for human developers, it is difficult to write an exactly correct program from scratch.

Novelty 2: Example Retrieval. To solve the above challenge, we propose *example retrieval*. It is inspired by the human developers' code reuse. In real-world scenarios, given a requirement, developers often refer to programs with similar requirements. They learn programming skills (e.g., APIs) or directly reuse relevant content from similar programs [23, 29].

Specifically, we use a *retriever* to search for programs with similar requirements. Considering the maximum input length of LLMs is limited (e.g., 1,024 tokens), the number of examples in a prompt is also limited, such as three examples. Thus, we further design a *selector* to select a set of programs from retrieved results as examples. The selector will filter out redundant programs and pick informative examples. Then, examples are inserted into prompts and teach LLMs how to implement code. We illustrate the example retrieval in Section 2 and describe the details in Section 3.2.

In conclusion, given a requirement, AceCoder generates a program in three steps:

- —*Example retrieval.* It uses a *retriever* and a *selector* to search for examples, i.e., <requirement, code> pairs.
- —*Prompt construction.* It uses an *analyzer* to convert example into <requirement, preliminary, code> triples. Then, it concatenates examples with the input requirement together to construct a prompt.
- —*Code generation.* It feeds the prompt into LLMs. By learning from examples, LLMs output an intermediate preliminary and then generate the source code.

We apply AceCoder to four representative LLMs, i.e., GPT-3.5 [32], CodeGeeX [51], CodeGen [30], and InCoder [13]. We conduct extensive experiments on three popular code generation benchmarks, i.e., MBPP [7], **Mostly Basic Java Problems (MBJP)** [6], and **Mostly Basic JavaScript Problems (MBJSP)** [6]. We employ Pass@k ($k = 1, 3, 5$) to measure the performance of different approaches. We obtain some findings from experimental results. ❶ AceCoder significantly outperforms existing prompting techniques. In terms of Pass@1, AceCoder outperforms the SOTA baseline—few-shot prompting by up to 56.4% in MBPP, 70.7% in MBJP, and 88.4% in MBJSP. The improvements prove the superiority of AceCoder in code generation. ❷ AceCoder substantially outperforms retrieval-based models. In terms of Pass@1, AceCoder outperforms the SOTA retrieval-based baseline by up to 13.1% in MBPP, 23.44% in MBJP, and 15.8% in MBJSP. ❸ AceCoder is effective in LLMs of different sizes. We apply AceCoder to three LLMs, which scale from 6B to 13B. In terms of Pass@1, AceCoder improves CodeGeeX-13B by up to 88.4%, CodeGen-6B by up to 65.5%, and InCoder-6B by up to 57.5%. ❹ Human evaluation shows that human developers prefer programs generated by AceCoder. Results show that AceCoder outperforms the SOTA baseline in multiple aspects, including correctness, code smell, and maintainability. ❺ We explore the contributions of different modules and discuss different designs for AceCoder. Results show that three modules are all necessary and our designs for three modules are superior to multiple alternates.

We summarize our contributions in this paper as follows:

- —We propose a novel prompting technique named AceCoder, for improving the performance of LLMs in code generation.
- —AceCoder contains two novel techniques (i.e., guided code generation and example retrieval) to alleviate two challenges (i.e., requirement understanding and code implementation) in code generation, respectively.
- —We apply AceCoder in three LLMs and conduct extensive experiments on three public benchmarks. Qualitative and quantitative experiments show that AceCoder significantly outperforms the SOTA baselines (e.g., CoT prompting, few-shot prompting).

(a) Input requirement:

```
# Write a python function to remove first and last
occurrence of a given character from the string
```

(b) Unit tests:

```
(1) ("hello world", "x") -> "hello world"
(2) ("hello world", "l") -> "helo word"
```

(c) Few-shot Prompting:

```
def remove_Occ(s, ch):
  return s.replace(ch, '', 1).replace(ch, '', -1)
```

Evaluation: (1) pass (2) fail

(d) AceCoder:

```
# ("test", "x") -> "test"
# ("test", "e") -> "tst"
# ("test", "t") -> "es"
def remove_Occ(s, ch):
  first_idx, last_idx = -1, -1
  for i in range(len(s)):
    if s[i] == ch:
      if first_idx == -1:
        first_idx = i
      last_idx = I
  # case 1: ch not in s
  if first_idx == -1 and last_idx == -1:
    return s
  # case 2: ch appears once in s
  elif first_idx == last_idx:
    return s[:first_idx] + s[first_idx+1:]
  # case 3: ch appears multi-times in s
  else:
    return s[:first_idx] + \
s[first_idx+1:last_idx] + s[last_idx+1:]
```

Evaluation: (1)(2) pass

Fig. 1. A motivating example of guided code generation.

Article Organization. Section 2 shows some examples to illustrate the motivations of AceCoder. Section 3 presents the details of our AceCoder. In Section 4 and Section 5, we conduct a large-scale study and evaluate the effectiveness of AceCoder. Section 6 discusses the differences between AceCoder and related works. Section 7 surveys related works about prompting techniques in code generation. Finally, Section 8 summarizes this article and points out some future directions.

2 Motivating Examples

In this section, we explain our motivations by some real cases.

Requirement Understanding → Guided Code Generation. Figure 1(a) and (b) show a requirement from a real-world benchmark [7] and its unit test for evaluation, respectively. We select Codex as the base model. Figure 1(c) shows a program generated by few-shot prompting. The program fails, as it ignores some essential scenarios in the requirement, such as `ch` appearing multiple times in `s`. It shows that comprehensively understanding the requirements is crucial to writing correct programs.

Thus, we propose guided code generation, which asks LLMs first to analyze the requirement and then generate code. Figure 1(d) shows a program generated by AceCoder. We consider test cases to be the intermediate preliminary. We can see that the generated test cases cover multiple

(a) Input requirement:

```
# Write a function to find sequences of lowercase
letters joined with an underscore.
```

(b) Unit tests:

```
(1) ('a_b_c') -> True
(2) ('a_ c_') -> False
```

(c) Few-shot Prompting:

```
def text_lowercase_underscore(text):
  words = text.split()
  for word in words:
    if word.islower() and '_' in word:
      return True
  return False
```

Evaluation: (1) pass (2) fail

(d) Retrieved Programs:

```
Program-1: find sequences of literals in a string.
def find_literals(text, pattern):
  match = re.search(pattern, text)
  (more lines...)
Programs-2&3: re.search(…)
Program-4: split a string at lowercase letters.
def split_upperstring(text):
  return re.findall("[a-z][^a-z]+", text)
```

(e) AceCoder:

```
def text_lowercase_underscore(text):
  import re
  patterns = '^[a-z]+_[a-z]+$'
  if re.search(patterns, text):
    return True
  else:
    return False
```

Fig. 2. A motivating example of example retrieval.

scenarios, e.g., boundary inputs (“test,” “e”). They further clarify the requirement and benefit the following code implementation. Based on the test cases, AceCoder generates a correct program, which considers three scenarios and gives solutions, respectively. The example shows that our guided code generation can help LLMs to analyze requirements and improve the functional correctness of code.

Code Implementation → Example Retrieval. After understanding the input requirement, implementing the code is challenging. It requires LLMs to use various algorithms or libraries. Figure 2(a) and (b) show a requirement from a real-world benchmark [7] and its unit test for evaluation, respectively. Figure 2(c) shows a failed program generated by few-shot prompting. The requirement is to find sequences of lowercase letters joined with an underscore (e.g., a_b_c). The failed program simply checks whether the sequence contains lowercase letters and underscores rather than joining. We suspect that the model does not know how to judge a string containing lowercase letters joined with an underscore.

To alleviate the above problem, we propose *example retrieval*. Our motivation is that human developers often learn programming skills from similar programs. Figure 2(d) shows a few programs with similar requirements. The retrieval metric is the BM25 score. We sort these programs in the descending order of BM25 scores. We can see that similar programs contain lots of relevant content (e.g., re.search), which benefits code implementation. Thus, we design a retriever to search for

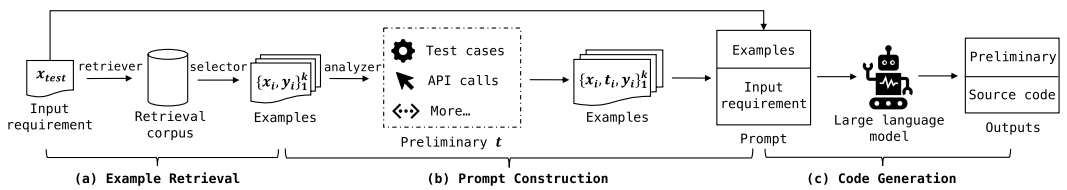

Fig. 3. An overview of AceCoder. Given a requirement, it selects examples from similar programs and constructs a prompt. LLMs first output an intermediate preliminary and then generate the source code. x, y, and t denote requirements, programs, and intermediate preliminaries, respectively.

similar programs as examples in prompts. We expect LLMs can learn from similar programs how to implement new programs.

Since the maximum input length of LLMs is usually limited (e.g., 1,024 tokens), the number of examples in a prompt is limited. Thus, we need to select a few programs from retrieved results as examples. A straightforward idea is to pick programs with the highest similarities. However, since each retrieval is independent, we find that retrieved results may contain redundant programs. For example, Program-1$2$3 in Figure 2(d) are redundant because they all present an API `re.search` that teaches how to search a pattern in the text. Program-4 contains a relevant regular expression, which tells how to design a pattern. Suppose the number of examples is 2. The examples will contain redundant programs (i.e., Program-1 and 2) and miss more informative Program-4.

Thus, we design a selector to filter out redundant programs in retrieved results. Suppose the number of examples is 2. In Figure 2(d), our selector will select Program-1 and Program-4 as examples. Figure 2(e) shows a program generated by AceCoder. It successfully learns how to write regular expressions from Program-4 and learns how to use `re.search` to find patterns from Program-1.

3 AceCoder

In this section, we propose a novel prompting technique for code generation, named AceCoder. We present an overview of AceCoder and then describe its details.

3.1 An Overview

Code generation aims to generate the source code y based on a natural language requirement x. AceCoder leverages LLMs to generate programs via prompting. Figure 3 shows an overview of AceCoder during inference. Given an input requirement x_{test}, AceCoder generates code in three steps.

- *Example Retrieval.* It uses a *retriever* and a *selector* to select k similar <requirement, code> pairs ($\{x_i, y_i\}_{i=1}^k$) from a retrieval corpus as examples.
- *Prompt Construction.* It employs an *analyzer* to convert examples into <requirement, preliminary, code> triples ($\{x_i, a_i, y_i\}_{i=1}^k$). A preliminary is a software artifact for clarifying the requirement, such as test cases. The examples are concatenated with the input requirement to construct a prompt.
- *Code Generation.* The prompt is fed into LLMs. By learning from examples, LLMs output an intermediate preliminary and then generate the code.

where x_i, y_i, a_i denote the requirement, the code, and the preliminary in ith example, respectively.

Input requirement:
```
# Write a function to find sequences of lowercase
letters joined with an underscore in a string.
```
Similar program-1:
```
# find sequences of literals in a string.
def find_literals(text, pattern):
  re.search(…)
```
Similar program-2:
```
# find sequences of an a followed by zero or more b's.
def text_match(text):
  re.search(…)
```
Similar program-3:
```
# find sequences of numbers containing a decreasing
trend or not.
def decreasing_trend(nums):
  re.search(…)
```
Similar program-4:
```
# split a text at lowercase letters.
def split_upperstring(text):
  return re.findall("[a-z][^a-z]*", text)
```

Fig. 4. A requirement and its similar programs.

3.2 Example Retrieval

As shown in Figure 3, the first step has two goals: (1) retrieve similar programs and (2) select a few examples from retrieved programs. We design a *retriever* and a *selector* to achieve these goals, respectively. The details of the two modules are shown as follows.

3.2.1 Retriever. Similar programs often have similar natural language requirements [AceCoder, 15, 23]. There, we take the input requirement as a query to search for similar requirements in a retrieval corpus. In this article, we consider the training data in experimental datasets as the retrieval corpus. Then, we extract the corresponding programs as similar programs.

Specifically, we leverage an open-source search engine named Lucene [3] to build our retriever and use the training data as a retrieval corpus. We employ the BM25 score [39] as the retrieval metric, which is widely used in previous studies [22, 46]. The BM25 score is a bag-of-words retrieval function and is used to estimate the lexical-level similarity of two sentences. The more similar the two sentences are, the higher the value of BM25 scores. In this article, the retriever outputs top-m similar programs based on the BM25 score.

The reason for choosing BM25+Lucene is that they can achieve good retrieval accuracy and have low complexity. Considering that the retrieval corpus is often large-scale, a lightweight retriever is closer to practical applications. In Section 5, we also explore other designs for the retriever and compare them to our design.

3.2.2 Selector. We can obtain top-m similar programs from the retriever. However, the maximum input length of LLMs (e.g., 1,024 tokens) and the inference budget are often limited. It leads that the number of examples (i.e., k) in a prompt is also limited (e.g., three examples). It is necessary to further select k programs from retrieved results as examples.

A straightforward idea is to pick top-k similar programs as examples. However, as the programs are scored independently, we find that retrieved results may contain redundant programs. Figure 4 shows a requirement and its similar programs. Similar programs are ranked by the BM25 score. We can see that top-3 programs are redundant, as all of them use an API (i.e., `re.search`) to find

Algorithm 1: The Algorithm of Our Selector

Inputs:
Input requirement x_{test}, similar programs $\{(x_i, y_i)\}_{i=1}^{m}$;
The number of examples $k, k <= m$, decay factor λ.
Outputs:
Selected examples T, $\{(x_i, y_i)\}_{i=1}^{k}$.

```
1:  T ← Empty Ordered List
2:  S ← Extract_ngrams_with_count(x_test)
3:  for i in {1, ··· , m} do
4:      Q[i] ← Extract_ngrams_with_count(x_i)
5:  end for
6:  while len(T) < k do
7:      for i in {1, ··· , m} do
8:          Score[i] ← Ngram_overlap_score(S, Q[i])
9:      end for
10:     j ← argmax(Score)
11:     T.append((x_j, y_j))
12:     matched_ngrams ← S ∩ Q[j]
13:     Q[j] ← ∅
14:     for i in {1, ··· , m} do
15:         for ngram ∈ match_ngrams do
16:             S[i][ngram]× = λ
17:         end for
18:     end for
19: end while
20: return T
```

sequences of a specific pattern. The Program-4's requirement contains a relevant regex expression. However, as Program-4 has fewer overlapping n-grams with the input requirement, it has a relatively low BM25 score. Obviously, directly selecting top-k (e.g., top-3) retrieved programs is unreasonable, as it will introduce redundant programs and ignore more informative Program-4.

In this article, we design a selector, which can filter out redundant programs in retrieved results. The algorithm of the selector is shown in Algorithm 1. We first extract all n-grams of the input requirement and all similar requirements (lines 2–5). In this article, n is set to 4 by default. Then, we calculate a recall-based ROUGE-n score between the input requirement and each similar requirement using the following equations (lines 7–9).

$$R_n = \frac{\sum_{n\_gram \in S \cap Q} S(n\_gram)}{\sum_{n\_gram \in S} S(n\_gram)} \tag{1}$$

$$Score = \exp\left(\frac{1}{n}\sum_{n} \log(R_n)\right). \tag{2}$$

We get a similar requirement with the maximum score and add its corresponding program to examples (lines 10–11). Then, the matched n-grams between the similar requirement and the input requirement are decayed by a factor λ. This process (lines 6–17) is repeated until the number of examples reaches the upper bound. The motivation for the decay is to filter out redundant programs, i.e., programs with the same matched n-grams. For example, in Figure 4, we first add Program-1 to

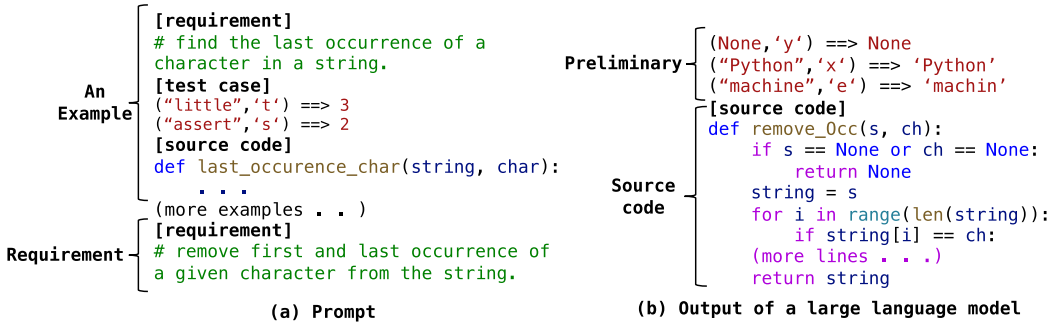

Fig. 5. Examples of our prompt and an LLM's output.

examples and then decay its matched n-grams (e.g., `find sequences of`). Subsequent programs with the same matched n-grams (i.e., Program-2 and Program-3) are considered redundant and will be ignored. Program-4 contains new matched n-grams (e.g., `lowercase letters`) and probably contains new information. Thus, Program-4 will obtain a higher score and is added to the examples.

By the above process, our selector filters out redundant programs and selects k similar programs as examples. In practice, m and k are small numbers, such as $m = 50$, $n = 3$. Thus, the time complexity of our selector is acceptable.

3.3 Prompt Construction

The goal of this step is to construct a prompt. As stated in Section 1, our guided code generation expects that LLMs can output an intermediate preliminary and then generate the final code. To achieve this goal, we design a special prompt consisting of triple examples (i.e., <requirement, preliminary, code>).

Specifically, we first use an analyzer to introduce preliminaries $\{t_i\}_{i=1}^{k}$ into selected examples $\{x_i, y_i\}_{i=1}^{k}$, obtaining triple examples $\{x_i, t_i, y_i\}_{i=1}^{k}$. The preliminary is a software artifact for clarifying requirements. Inspired by test-driven development [8], this article considers test cases as the preliminary by default. We also explore other choices (e.g., APIs, method signature) in Section 5. Then, we concatenate these triple examples with the input requirement to construct a prompt.

Figure 5(a) shows an example of our prompt. The prompt begins with several examples and ends with a new requirement. `[requirement]`, `[test case]`, and `[source code]` are special tags that mark different parts in a triple.

We assume that test cases of examples are available. We think this assumption is acceptable. The reasons are two-fold. First, there are many public code generation datasets containing test cases, e.g., MBPP [7] (474 samples), APPS [16] (5,000 samples), and CodeContest [25] (13,328 samples). We can extract training data from these datasets and construct a retrieval corpus. Second, test-driven software development is popular in real-world scenarios. We can mine software repositories from open-source communities (e.g., GitHub [2]) and extract code snippets equipped with test cases.

3.4 Code Generation

In this step, we leverage an LLM to generate code based on the prompt. Following previous studies [12, 13, 30, 51], we view the LLM as a black-box generator and use it to complete the prompt. By learning from examples in the prompt, LLMs will first output a preliminary (e.g., test cases) and then generate code based on the preliminary and the requirement.

Figure 5(b) shows an output of an LLM—CodeGeeX [51]. We can see that CodeGeeX first generates some test cases and then implements a Python function. The test cases provide lots of valuable information (e.g., input–output formats, invalid inputs) and guide the subsequent code generation.

Table 1. Statistics of the Datasets in Our Experiments

Statistics	MBPP	MBJP	MBJSP
Language	Python	Java	JavaScript
# Train	384	383	383
# Dev	90	90	90
# Test	500	493	493
Avg. tokens in requirement	16.50	16.71	16.53
Avg. tokens in code	92.68	247.79	100.75

4 Study Design

To assess AceCoder, we perform a large-scale study to answer six research questions. In this section, we describe the details of our study, including datasets, evaluation metrics, baselines, and base LLMs.

4.1 Research Questions

Our study aims to answer the following research questions (RQs).

RQ1: How does AceCoder perform compared to existing prompting techniques? This RQ aims to validate that AceCoder has higher accuracy than existing prompting techniques in code generation. We apply AceCoder and baselines to three LLMs and measure their accuracy on three code generation benchmarks. The evaluation metric is Pass@K.

RQ2: How does AceCoder perform compared to retrieval-based models? AceCoder retrieves similar programs as examples in prompts. Some existing studies [19, 35] also introduce information retrieval to augment code generation. In this RQ, we compare AceCoder to these retrieval-based models. The evaluation metric is Pass@K.

RQ3: Do human developers prefer code generated by AceCoder? The ultimate goal of code generation is to assist human developers in writing code. In this RQ, we hire 10 developers (including industry employees and academic researchers) to review the code generated by AceCoder and baselines manually. We measure the quality of code in three aspects, including correctness, code smell, and maintainability.

RQ4: What are the contributions of different modules in AceCoder? AceCoder contains three modules, i.e., a retriever, a selector, and an analyzer. This RQ is designed to analyze the contributions of three modules to the performance. We select a base model, gradually introduce three modules, and observe the fluctuations in accuracy.

RQ5: What are the better designs for three modules? This RQ aims to validate the superiority of our designs for three modules in AceCoder. Specifically, we explore multiple designs for three modules and compare them to our designs.

4.2 Evaluation Datasets and Metrics

4.2.1 Datasets. We conduct experiments on three public code generation benchmarks, including the MBPP in Python, MBJP in Java, and MBJSP in JavaScript. The statistics of the datasets are shown in Table 1. The details of the datasets are described as follows.

— *MBPP* [7] contains 974 real-world programming problems that are constructed by crowdsourcing. Each problem contains a natural language requirement, a single Python function, and three test cases.

—*MBJP* [6] and *MBJSP* [6] both contain 966 crowd-sourced programming problems in Java and JavaScript, respectively. Each problem consists of a natural language requirement, an individual function, and three test cases.

4.2.2 Metrics. Following previous code generation studies [12, 13, 30, 51], we employ Pass@k as our evaluation metric. Specifically, we generate k programs for each requirement. A requirement is considered solved if any generated programs pass all test cases. We compute the percentage of solved requirements in total requirements as Pass@k. In this article, k is set to 1, 3, and 5.

We notice that previous studies [15, 45] also use some match-based metrics (e.g., **Bilingual Evaluation Understudy (BLEU)** [34]). These metrics are initially designed for natural language generation and are poor in measuring the functionality of programs [12]. Thus, we omit them in experiments.

4.3 Comparison Baselines

This article is to propose a new prompting technique for code generation. Thus, we select three existing prompting techniques as baselines.

—*Zero-shot prompting* [12, 30] directly feeds the input requirement into LLMs. Then, it extracts the code from LLMs' outputs.
—*Few-shot prompting* [12] randomly selects several (3 in this article) <requirement, code> pairs from the training data as examples and constructs a prompt, which is fed into an LLM. Then, it extracts the code from LLMs' outputs. The prompt of few-shot prompting is available in our replication package [AceCoder].
—*CoT prompting* [48] is a variant of few-shot prompting. CoT prompting uses several (3 in this article) <requirement, intermediate steps, code> triples as examples and constructs a prompt. Based on the prompt, LLMs first generate a series of intermediate steps and then output the code. The prompt of CoT prompting is available in our replication package [AceCoder].

AceCoder retrieves similar programs to assist LLMs in generating code. Some studies also introduce information retrieval to augment code generation. We compare AceCoder to these retrieval-based models.

—REDCODER [35] retrieves similar programs and fine-tunes a pre-trained model—PLBART [5] to generate code based on the requirement and similar programs.
—*Jigsaw* [19] searches for similar programs from API documentation and insert them into the prompts.

4.4 Base LLMs

We select three open-source LLMs as base models. The details of the base models are shown as follows.

—*GPT-3.5* [32] is a variant of gpt-3 [9] through the **reinforcement learning with human feedback (RLHF)**. The RLHF can improve models' instruction-following capabilities and avoid the generation of harmful or toxic content. In this article, we utilize the `gpt-3.5-turbo-0301` version.
—*CodeGeeX* [51] is a multilingual LLM for source code with 13 billion parameters. CodeGeeX is pre-trained with a large corpus of more than 20 programming languages (e.g., Python, Java, and JavaScript). We download the model weight and run CodeGeeX following official instructions.